

MySQL数据库运维手册

1. 安装与基础安全规范

1.1 版本选择与环境

生产环境建议使用MySQL 5.7或8.0系列的最新稳定版。MySQL 8.0引入了窗口函数、CTE（通用表表达式）和资源组管理，显著提升了复杂查询和资源控制能力。操作系统优先选择Linux（如CentOS/RHEL或Ubuntu/Debian），文件系统推荐XFS或Ext4。

1.2 安装后安全加固

安装完成后，必须执行安全初始化脚本，这是生产环境的最低安全基线：

```
sudo mysql_secure_installation
```

该脚本强制要求设置root密码、移除匿名用户、禁止root远程登录并删除test数据库。

1.3 账号权限管理

遵循最小权限原则，为不同的应用创建独立的数据库账号，严禁应用使用root账户连接数据库。

```
-- 创建只允许特定网段访问的业务账号
CREATE USER 'app_user'@'10.0.0.%' IDENTIFIED BY 'StrongPassword123!';
-- 授予业务库的增删改查权限
GRANT SELECT, INSERT, UPDATE, DELETE ON app_db.* TO 'app_user'@'10.0.0.%';
-- 回收不必要的全局权限
REVOKE DROP, CREATE ON *.* FROM 'app_user'@'10.0.0.%';
FLUSH PRIVILEGES;
```

2. 配置调优

2.1 关键内存与I/O参数

合理的参数配置是性能的基础。以下配置基于16GB以上内存的专用服务器。

```
[mysqld]
# InnoDB缓冲池（建议物理内存的50%-70%）
innodb_buffer_pool_size = 8G
# 缓冲池实例数（减少内部锁争用，建议1-64）
innodb_buffer_pool_instances = 8

# 重做日志（写入高峰场景建议1G-2G）
innodb_log_file_size = 1G
# 日志缓冲
innodb_log_buffer_size = 16M

# 刷盘策略（双1标准，保证ACID）
innodb_flush_log_at_trx_commit = 1
# 同步二进制日志（主从环境强烈建议开启）
sync_binlog = 1

# I/O线程与容量（SSD环境下调整）
innodb_io_capacity = 2000
innodb_io_capacity_max = 4000
# 绕过操作系统缓存，减少双重缓存
```

```
innodb_flush_method = O_DIRECT

# 连接数（根据应用线程池设置）
max_connections = 500
thread_cache_size = 100

# 字符集（必须使用utf8mb4以支持emoji）
character-set-server = utf8mb4
collation-server = utf8mb4_unicode_ci
```

2.2 Linux系统层面优化

修改 /etc/sysctl.conf 以优化网络和内存管理：

```
# 端口范围与网络缓存
net.ipv4.ip_local_port_range = 15000 65000
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
# 内存交换倾向（降低swappiness倾向于回收页缓存而非交换内存）
vm.swappiness = 1
```

NUMA策略：在NUMA架构下，建议关闭MySQL的numa特性或使用 `numactl --interleave=all` 启动MySQL，防止内存跨节点访问性能下降。

3. 日常运维与监控

3.1 定期维护任务

定期维护是保障数据库长期稳定运行的关键。

任务类型	操作命令/方法	频率建议
表碎片整理	<code>OPTIMIZE TABLE table_name;</code> (InnoDB会重建表)	每月/季度，或删除大量数据后
更新统计信息	<code>ANALYZE TABLE table_name;</code>	实例升级或数据分布变化较大时
日志轮转	配置 logrotate 管理 <code>slow-query.log</code> 和 <code>error.log</code>	每日
碎片检查	<code>SELECT ... FROM information_schema.TABLES WHERE DATA_FREE > 0</code>	每周巡检

3.2 慢查询分析

开启慢查询日志是发现性能问题的核心手段。

```
-- 开启慢查询记录（执行时间超过2秒的SQL）
SET GLOBAL slow_query_log = ON;
SET GLOBAL long_query_time = 2;
-- 记录未使用索引的查询
SET GLOBAL log_queries_not_using_indexes = ON;
```

使用 `pt-query-digest` 工具定期分析慢查询日志，定位TOP SQL。

3.3 监控重点指标

通过 `SHOW STATUS` 和 `Performance_Schema` 关注以下关键指标：

- **缓冲池命中率**： `Innodb_buffer_pool_reads` 和 `Innodb_buffer_pool_read_requests` 。理想值应大于99%。
 - **锁等待**： `Innodb_row_lock_waits` 和 `Innodb_row_lock_time_avg` 。持续增长通常意味着存在行锁争用。
 - **连接数**： `Threads_connected` 和 `Max_used_connections` 。监控是否接近 `max_connections` 上限。
-

4. 备份与恢复策略

4.1 备份方案选型

- **逻辑备份 (mysqldump)** ：适用于数据量较小 (<100GB) 或结构迁移场景。使用 `--single-transaction` 参数保证InnoDB备份的一致性，不锁表。
- **物理备份 (XtraBackup)** ：适用于生产环境大库 (TB级别) ，支持热备份和增量备份，恢复速度快。

4.2 恢复演练

备份仅是数据安全的一部分，**恢复验证**才是关键。建议每季度至少进行一次完整的恢复演练。

- 常规恢复： `mysql -u root -p < backup.sql`
- 利用二进制日志进行时间点恢复 (Point-In-Time Recovery) ，以消除误操作影响：

```
mysqlbinlog --stop-datetime="2023-10-01 12:00:00" binlog.000001 | mysql -u root -p
```

5. 高可用架构

5.1 异步/半同步复制

标准的主从复制架构用于读写分离和容灾。

- **主库**：开启 `binlog` ，设置 `server-id` ，创建复制账号。
- **从库**：设置 `read_only` ，通过 `CHANGE MASTER TO` 建立连接。

5.2 高可用方案

- **MHA/Orchestrator**：管理主从切换，通常配合VIP (虚拟IP) 或DNS使用。
 - **InnoDB Cluster**：MySQL官方推荐方案，基于Group Replication (组复制) 和MySQL Router中间件，实现自动故障转移和读写分离。
-

6. 紧急故障处理

6.1 连接数爆满

若应用异常或突发流量导致连接数耗尽，普通用户无法登录，可使用 `extra_port` 预留通道登录进行杀进程。

```
-- 在配置文件中预留端口
[mysqld]
extra_port = 13306
-- 登录后批量杀掉空闲连接
SELECT CONCAT('KILL ', id, ';') FROM information_schema.processlist WHERE
Command='Sleep' AND Time > 1000;
```

6.2 表损坏与数据救援

- **MyISAM**: 使用 `REPAIR TABLE table_name;` 尝试修复。
- **InnoDB**: 若InnoDB表空间损坏且无法启动, 可在配置文件中设置 `[mysqld]`
`innodb_force_recovery = 1` (数值1-6) 尝试导出数据。数值越大表示禁用越多的后台功能, 通常设为4或6是为了 `SELECT` 导出数据。

7. 安全加固清单

1. **传输加密**: 开启 `require_secure_transport` 强制使用SSL/TLS连接。
2. **密码审计**: 开启 `validate_password` 组件强制密码复杂度策略。
3. **权限审计**: 定期使用 `SHOW GRANTS FOR 'user'@'host';` 审计权限, 删除僵尸账号 (host 为 % 且无用的账号)。
4. **文件安全**: 确保 `datadir` 目录权限为750 (属主mysql), 防止未经授权的操作系统用户读取数据库文件。